

WEEK-6: KAFKA PRODUCER AND CONSUMER

Experiment

Develop a Java program to create a Kafka consumer, subscribe to a topic, and consume messages.

Step-1: Create Project Structure

Create the project:

~/Desktop/week-6/kafka-producer-demo/

Project structure:

```
kafka-producer-demo/  
├── pom.xml  
├── docker-compose.yml  
├── src/  
│   ├── main/  
│   │   └── java/  
│   │       ├── SimpleKafkaProducer.java  
│   │       └── SimpleKafkaConsumer.java
```

Open Terminal and execute:

```
cd ~/Desktop  
mkdir -p week-6/kafka-producer-demo/src/main/java  
cd week-6/kafka-producer-demo
```

Step-2: Create SimpleKafkaProducer.java

Save inside:

```
src/main/java/SimpleKafkaProducer.java  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.ProducerRecord;  
import org.apache.kafka.clients.producer.RecordMetadata;
```

```
import java.util.Properties;  
import java.util.Scanner;
```

```
public class SimpleKafkaProducer {  
  
    public static void main(String[] args) {  
  
        Properties props = new Properties();
```

```

props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer",
    "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer",
    "org.apache.kafka.common.serialization.StringSerializer");

try (KafkaProducer<String, String> producer =
    new KafkaProducer<>(props);
    Scanner scanner = new Scanner(System.in)) {

    String topicName = "test-topic";

    System.out.println("Connected to Kafka topic: " + topicName);
    System.out.println("Type messages to send (type 'exit' to quit):");

    while (true) {

        System.out.print("Message: ");
        String message = scanner.nextLine();

        if ("exit".equalsIgnoreCase(message)) {
            break;
        }

        ProducerRecord<String, String> record =
            new ProducerRecord<>(topicName, message);

        RecordMetadata metadata = producer.send(record).get();

        System.out.printf(
            "Sent message to partition %d with offset %d%n",
            metadata.partition(),
            metadata.offset());
    }

    System.out.println("Producer closed.");

} catch (Exception e) {
    e.printStackTrace();
}
}

```

Step-3: Create SimpleKafkaConsumer.java

Save inside:

src/main/java/SimpleKafkaConsumer.java

```

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class SimpleKafkaConsumer {

    public static void main(String[] args) {

        Properties props = new Properties();

        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "my-consumer-group");
        props.put("key.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("auto.offset.reset", "earliest");

        KafkaConsumer<String, String> consumer =
            new KafkaConsumer<>(props);

        consumer.subscribe(Collections.singletonList("test-topic"));

        System.out.println("Subscribed to topic: test-topic");
        System.out.println("Waiting for messages...");

        try {
            while (true) {

                ConsumerRecords<String, String> records =
                    consumer.poll(Duration.ofMillis(1000));

                for (ConsumerRecord<String, String> record : records) {

                    System.out.printf(
                        "Consumed message: %s (partition=%d, offset=%d)%n",
                        record.value(),
                        record.partition(),
                        record.offset());
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
    
```

```
        consumer.close();
    }
}
}
```

Step-4: Configure pom.xml

Open pom.xml and make sure the following dependencies are present:

```
<dependencies>

    <dependency>
        <groupId>org.apache.kafka</groupId>
        <artifactId>kafka-clients</artifactId>
        <version>3.5.0</version>
    </dependency>

    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>2.0.9</version>
    </dependency>

    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>2.0.9</version>
    </dependency>

</dependencies>

<build>
    <plugins>

        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
            <version>3.1.0</version>
        </plugin>

    </plugins>
</build>
```

Step-5: Compile the Project

From the project folder:

```
mvn clean compile
```

If there are no errors, continue.

Step-6: Start Kafka and Zookeeper

Make sure docker-compose.yml is available in the project folder.

Start the containers:

```
docker compose up -d
```

Check the running containers:

```
docker ps
```

Kafka must be accessible through:

```
localhost:9092
```

Step-7: Run Kafka Consumer

Terminal-1

Open a new terminal:

```
cd ~/Desktop/week-6/kafka-producer-demo
```

Run:

```
mvn exec:java -Dexec.mainClass=SimpleKafkaConsumer
```

Expected output:

```
Subscribed to topic: test-topic  
Waiting for messages...
```

Keep Terminal-1 running.

Step-8: Run Kafka Producer

Terminal-2

Open another terminal:

```
cd ~/Desktop/week-6/kafka-producer-demo
```

Run:

```
mvn exec:java -Dexec.mainClass=SimpleKafkaProducer
```

Expected output:

Connected to Kafka topic: test-topic

Type messages to send (type 'exit' to quit):

Message:

Enter:

Hello Kafka

Expected Producer output:

Sent message to partition 0 with offset 1

At the same time, **Terminal-1 (Consumer)** should display:

Consumed message: Hello Kafka (partition=0, offset=1)

Step-9: Test More Messages

In Terminal-2:

Message: Welcome to Kafka

Message: This is Week-6 Experiment

Message: Kafka Consumer Test

Verify that all messages are displayed in Terminal-1.

To stop the Producer:

Message: exit

Expected:

Producer closed.

Step-10: Stop Kafka

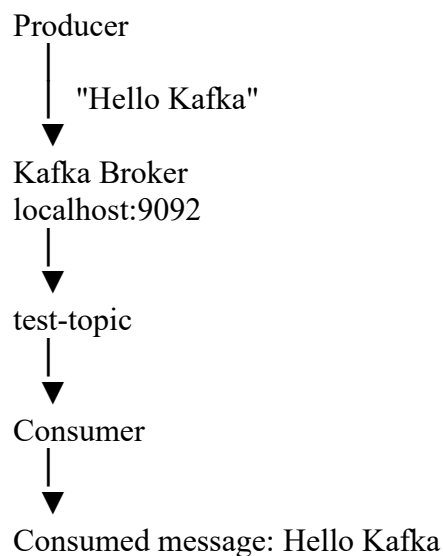
After completing the experiment:

```
docker compose down
```

ALL COMMANDS – QUICK EXECUTION ORDER

- # 1. Go to project
`cd ~/Desktop/week-6/kafka-producer-demo`
- # 2. Compile
`mvn clean compile`
- # 3. Start Kafka + Zookeeper
`docker compose up -d`
- # 4. Check containers
`docker ps`
- # 5. Run Consumer – Terminal 1
`mvn exec:java -Dexec.mainClass=SimpleKafkaConsumer`
- # 6. Run Producer – Terminal 2
`mvn exec:java -Dexec.mainClass=SimpleKafkaProducer`
- # 7. After completing the experiment, stop Kafka
`docker compose down`

Expected Flow



Result:

The Java Kafka Producer successfully sends messages to the test-topic, and the Kafka Consumer subscribes to the topic and successfully consumes the messages.